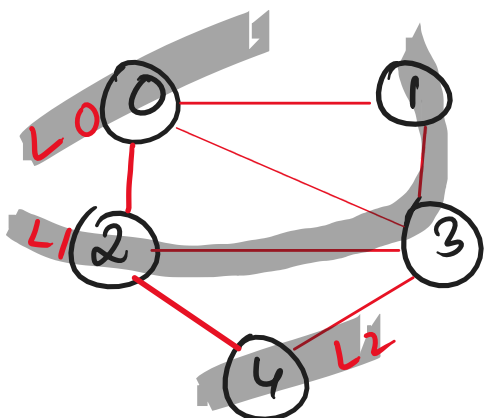


Graph Traversal : →

Intuition Or Idea Behind ^(queue) BFS (Level Order Traversal) : →

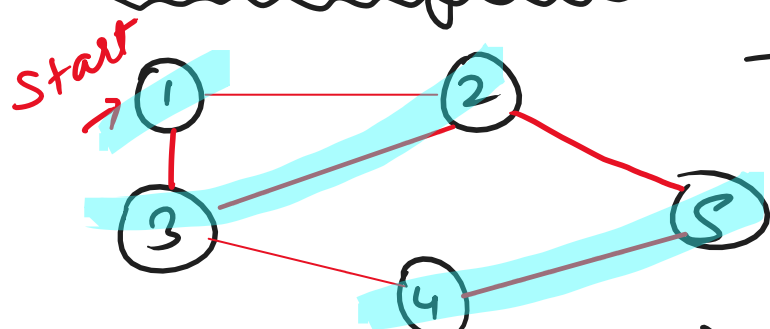


There will be an input given always :

The Starting Node
Say start = 0th Node

Prerequisites : → ① Visited Array
② Queue (FIFO)

BFS Dry Run : → <List> Output : → 1, 2, 3, 5, 4



→ * Who are your neighbours:

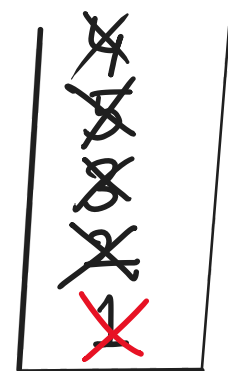
List.add (q.poll)

Adj List :

→ 1 → 2, 3
→ 2 → 1, 5
→ 3 → 1, 4
→ 4 → 3, 5
→ 5 → 2, 4

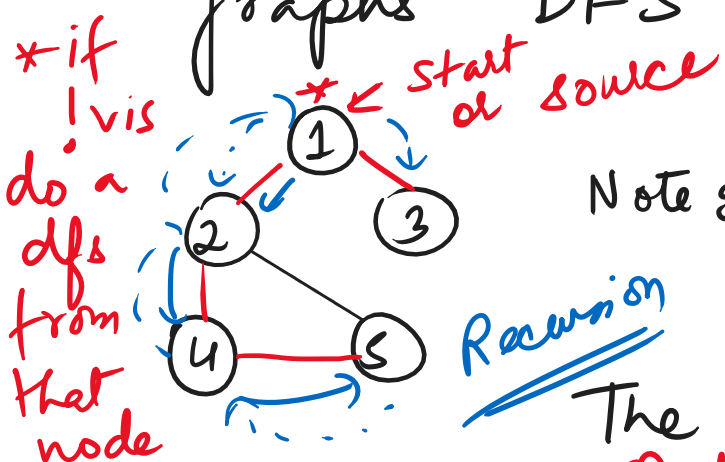
Visited Array :

T	T	T	T	T
☒	☒	☒	☒	☒
1	2	3	4	5



queue
(empty)

Graphs DFS Traversal (Very Easy)



Note : There can be n number of answers depending on the Starting node or vertex.

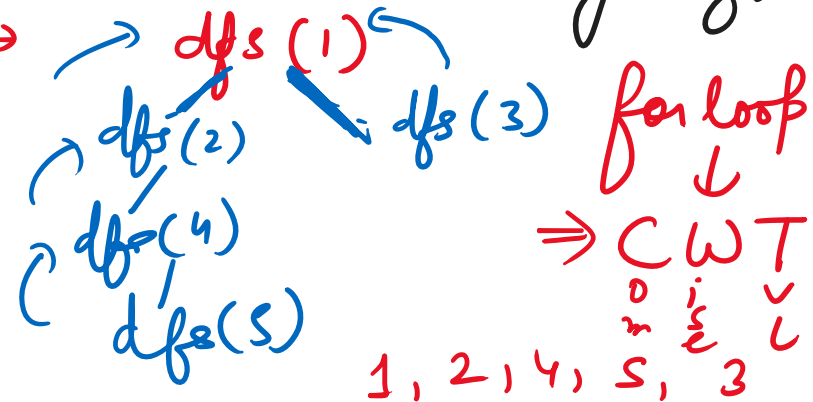
The start node will be always given

Adjacency List :

→ 1 → 2, 3
→ 2 → 1, 4, 5
→ 3 → 1, 5
→ 4 → 2, 5
→ 5 → 2, 4

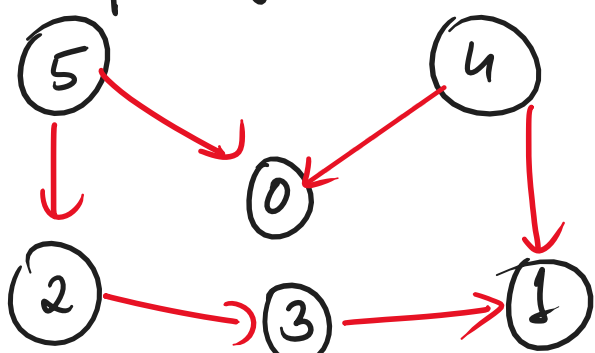
Output : →

1	☒	T
2	☒	T
3	☒	T
4	☒	T
5	☒	T



Topological Sort Algorithms : → (DAG)

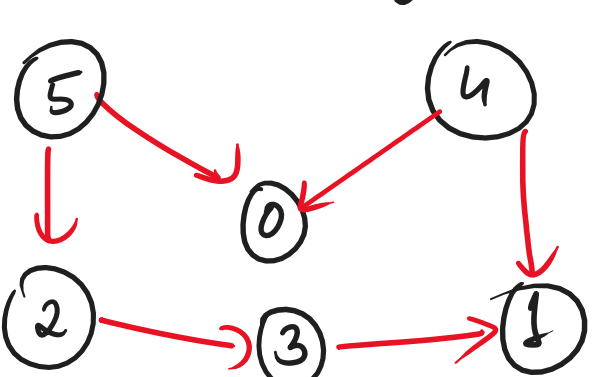
Directed Acyclic Graph



Ans 1 : 5 4 2 3 1 0
Ans 2 : 4 5 2 3 1 0
Edges : 5 → 0, 4 → 0, 2 → 3, 3 → 1
(6 edges)

Definition : → Linear Ordering of nodes/vertices such that if there is an edge from node u to node v, in the linear ordering, "u always comes before v"
(Linear Ordering)

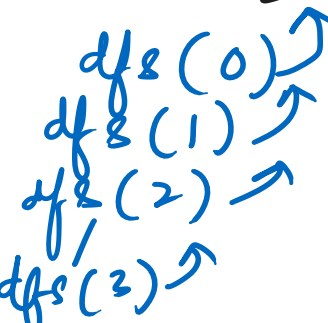
Topo Sort Algorithm : → (DFS Algo)



O/p : → 5, 4, 2, 3, 1, 0

* Note : We do a component-wise traversal (loop from 0 to n-1)
(0, 1, 2, 3, 4, 5) loop ends
dfs(i)

vis : 0 1 2 3 4 5



Adj List

0 →
1 →
2 → 3
3 → 1
4 → 0, 1
5 → 0, 2
dfs = finish
enter stack

Stack (LIFO)